

Descripción del caso

Una clínica local necesita modernizar su sistema de gestión de citas médicas. Actualmente, la información de sus pacientes, el personal médico, las citas programadas y las cancelaciones de último minuto se manejan en archivos de texto separados. Se le ha encomendado desarrollar un programa en C++ basado estrictamente en el uso de punteros genéricos (void* y void**) y asignación de memoria dinámica para consolidar esta información.

El sistema debe ser capaz de:

1. Leer los registros de pacientes y doctores, almacenándolos en memoria mediante la técnica de **asignación dinámica por incrementos**.
2. Asignar el historial de citas y las solicitudes de cancelación al expediente individual de cada paciente.
3. Procesar las solicitudes de cancelación cruzando los datos y actualizando el estado de las citas afectadas.
4. Emitir un reporte final detallado y liberar el 100% de la memoria dinámica utilizada antes de finalizar la ejecución.

Para ello, cuenta con los siguientes archivos en formato CSV:

pacientes.csv (DNI, Nombre, Edad, Género)

10000001,Juan Perez,30,M

10000002,Maria Gomez,25,F

doctores.csv (DNI, Nombre, Código de Especialidad)

20000001,Miguel Garcia,0

20000002,Laura Martinez,2

citas.csv (Fecha [AAAA-MM-DD], Hora [HH:MM], DNI Paciente, DNI Doctor, Estado) *Nota: El estado 1 indica cita RESERVADA y 0 indica cita CANCELADA.*

2026-02-01,08:00,10000001,20000001,0

2026-02-01,08:30,10000002,20000004,1

cancelaciones.csv (Fecha [AAAA-MM-DD], Hora [HH:MM], DNI Paciente, DNI Doctor)

2026-02-01,08:00,10000001,20000001

Con esta información, elabore un proyecto cuya función main contenga exactamente el siguiente código:

```
#include "funciones.h"
#include <fstream>
using namespace std;

int main() {
    void* clinica;

    cargarDatos(clinica,
                "pacientes.csv",
                "doctores.csv",
                "citas.csv",
                "cancelaciones.csv");

    cancelarCitas(clinica);

    ofstream reporte("reporte-clinica.txt", ios::out);
    imprimirReporte(reporte, clinica);

    liberarClinica(clinica);

    return 0;
}
```

Para estandarizar las posiciones de los arreglos de punteros genéricos, debe utilizar obligatoriamente el siguiente archivo **enum.h**

Parte 1: Carga de Datos Base (6 puntos)

Implemente la función cargarDatos y sus funciones auxiliares para inicializar la estructura principal clinica (la cual contendrá 2 punteros: uno al arreglo de pacientes y otro al de doctores).

1. **cargarPacientes y cargarDoctores:** Deberá leer los archivos respectivos y construir arreglos dinámicos utilizando el método de **asignación de memoria por incrementos** (ej. incrementos de 5 en 5).
2. **Construcción de Registros:** Cada paciente se almacenará como un arreglo void** de 6 posiciones, donde la fecha y hora de los eventos futuros (pacCitas y pacCancelaciones) deben inicializarse en nulo. Las fechas y horas de los archivos se deben unificar en enteros (Ej. 2026-02-01 será el entero 20260201 y 08:30 será el entero 830).

Parte 2: Consolidación de Historiales (6 puntos)

Una vez que los pacientes y doctores existen en la memoria, complete la carga de información leyendo los archivos de transacciones.

1. **cargarCitas y cargarCancelaciones:** Al leer una cita o una cancelación, usted debe buscar al paciente correspondiente por su DNI. Si el paciente existe, deberá agregar el registro de la cita o cancelación a los arreglos `pacCitas` o `pacCancelaciones` correspondientes.
2. **Asignación Exacta:** A diferencia de los arreglos principales que usan capacidad por incrementos, los arreglos internos de citas y cancelaciones de cada paciente deben dimensionarse al tamaño **exacto** requerido utilizando una función auxiliar (ej. `agregarRegistroExacto`). Si el paciente no se encuentra en el sistema, la memoria reservada para esa cita/cancelación debe ser liberada inmediatamente para evitar fugas de memoria.

Parte 3: Procesamiento Lógico (4 puntos)

Implemente la función `cancelarCitas(void* clinica)`.

Esta función recorrerá el arreglo de pacientes. Por cada paciente, evaluará su lista de cancelaciones registradas y cruzará la información con su lista de citas programadas. Si una cancelación coincide **exactamente** en Fecha, Hora y DNI del Doctor con una cita existente, el estado de dicha cita (`citaEstado`) deberá modificarse a 0 (Cancelada).

Parte 4: Reporte y Liberación (4 puntos)

1. **imprimirReporte:** Genere un archivo de texto llamado `reporte-clinica.txt` mostrando el historial de atenciones de todos los pacientes. Deberá mostrar la información de la cita, cruzando el DNI del doctor con el arreglo maestro de doctores para imprimir su nombre y traducir su especialidad (Ej. 0: CARDIOLOGIA, 1: DERMATOLOGIA, etc.).
2. **liberarClinica:** Desarrolle un esquema de funciones destructoras (`liberarPacientes`, `liberarPaciente`, `liberarDoctores`, etc.) que garantice la devolución total de la memoria dinámica al sistema operativo. Un programa con fugas de memoria (*memory leaks*) perderá el puntaje íntegro de esta sección.